

Distributed Systems

Lecture6: Layered-system Architectures

Mohammad Reza Shakournia

shakournia@ut.ac.ir

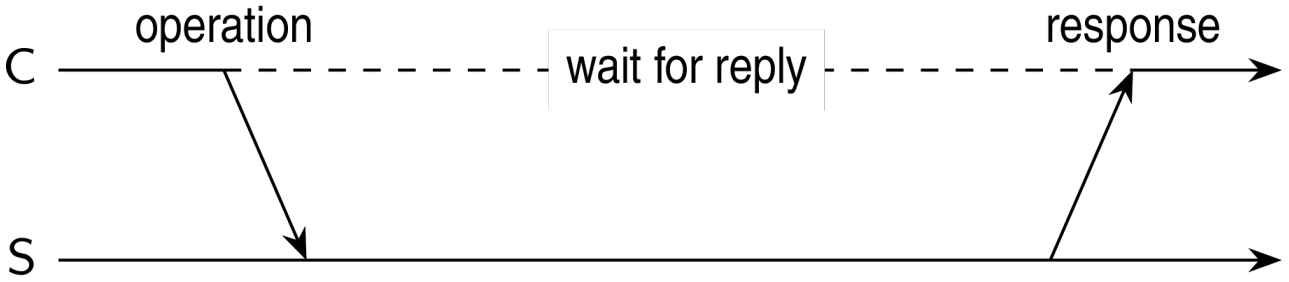
School of Electrical and Computer Engineering,
University of Tehran,
Tehran, Iran

Layered-system Architectures

- We can organize a distributed systems by considering where to place software components are placed,.
- Deciding on software components, their interaction, and their placement leads to an instance of a **software architecture**, also known as a **system architecture**.
- In layered-system architectures we will discuss:
  - Simple client-server architecture
  - Multitiered Architectures

Simple Client-server Architecture

- In the basic client-server model, we have two groups of processes:
  - A **server** is a process implementing a specific service. (a file system service or a database service)
  - A **client** is a process that requests a service from a server by sending it a request and subsequently waiting for the server's reply.



Simple Client-server Architecture

- Communication between a client and a server can be a simple connectionless protocol (in LAN).
- When a client requests a service:
  - It packages a message for the server,
  - identifying the service it wants,
  - the necessary input data,
  - sent the message to the server.
- The server:
  - will always wait for an incoming request,
  - process it,
  - package the results in a reply message,
  - sent to the client.

Connectionless Protocol

- As long as messages do not get lost or corrupted, connectionless has advantage, otherwise it is failure-prone.
- In this case, the client could resend the request when no reply message comes in.
  - the client cannot detect whether the original request message was lost,
  - or that transmission of the reply failed.
- If the reply was lost, then resending a request may result in **performing the operation twice**.
  - *“transfer $10,000 from my bank account,” -> report an error!*
  - *“tell me how much money I have left,” -> resend!*
- **As an alternative, many client-server systems use a reliable connection oriented protocol.**

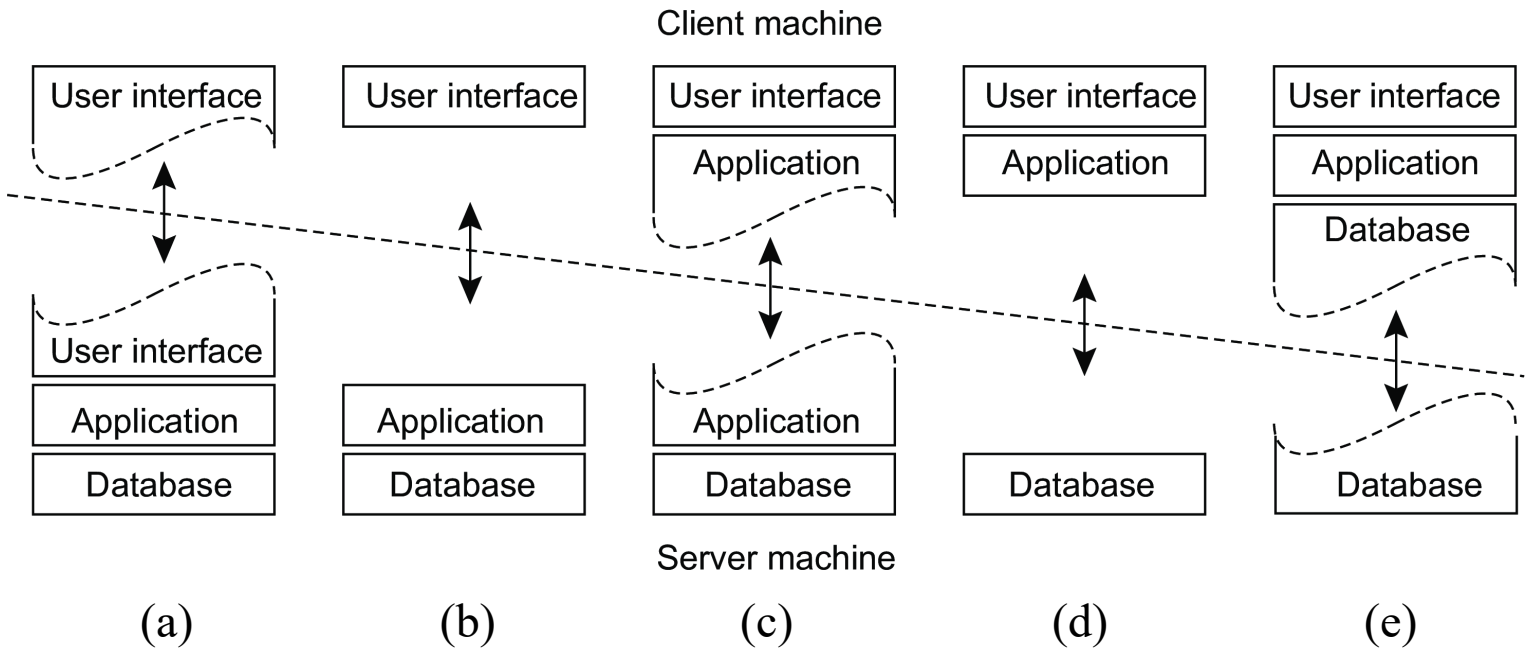
Connection-oriented Protocol

- Whenever a client requests a service, it first sets up a connection to the server before sending the request.
- The server generally uses that same connection to send the reply message, after which the connection is torn down.
- It works perfectly fine in wide-area systems.
- The trouble may be that setting up and tearing down a connection is relatively costly, especially when the request and reply messages are small.
- One of the main issues of the client-server architecture is that, there is often no clear distinction between a client and a server.

Multitiered Architectures

- The simplest organization to distribute a client-server application is to have only two types of machines:
  - A **client machine** containing only the programs implementing (part of) the user-interface level.
  - A **server machine** containing the rest, that is, the programs implementing the processing and data level.
- In this organization everything is handled by the server.
- The client is essentially no more than a dumb terminal, possibly with only a convenient graphical interface.
- We could distribute a client-server application across different machines by distributing these three layers:
  1. a user-interface layer,
  2. a processing layer,
  3. a data layer.

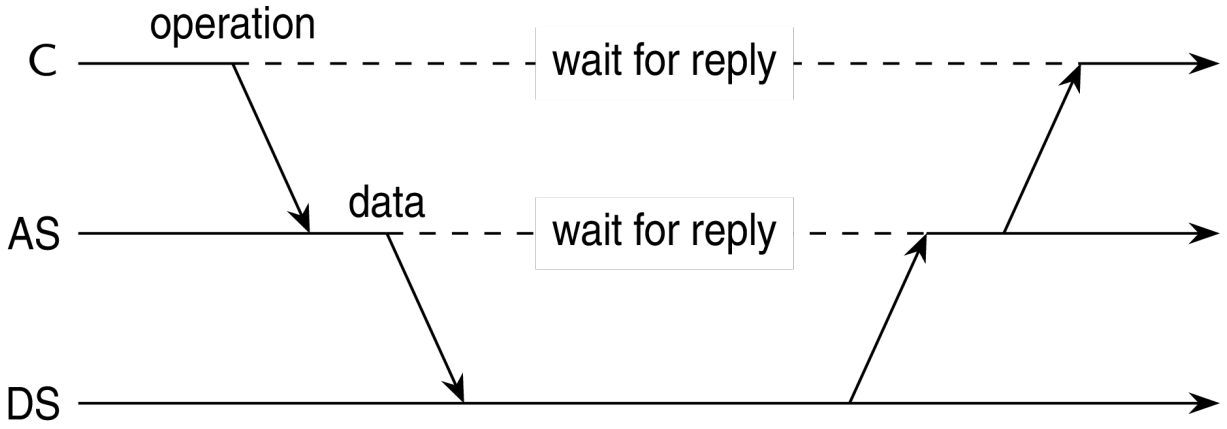
Two-tiered architecture



- The organizations (d) and (e) are particularly popular.
- Essentially, most of the application is running on the client machine, but all operations on files or database entries go to the server.(Bank Applications)

Three-tiered Architecture

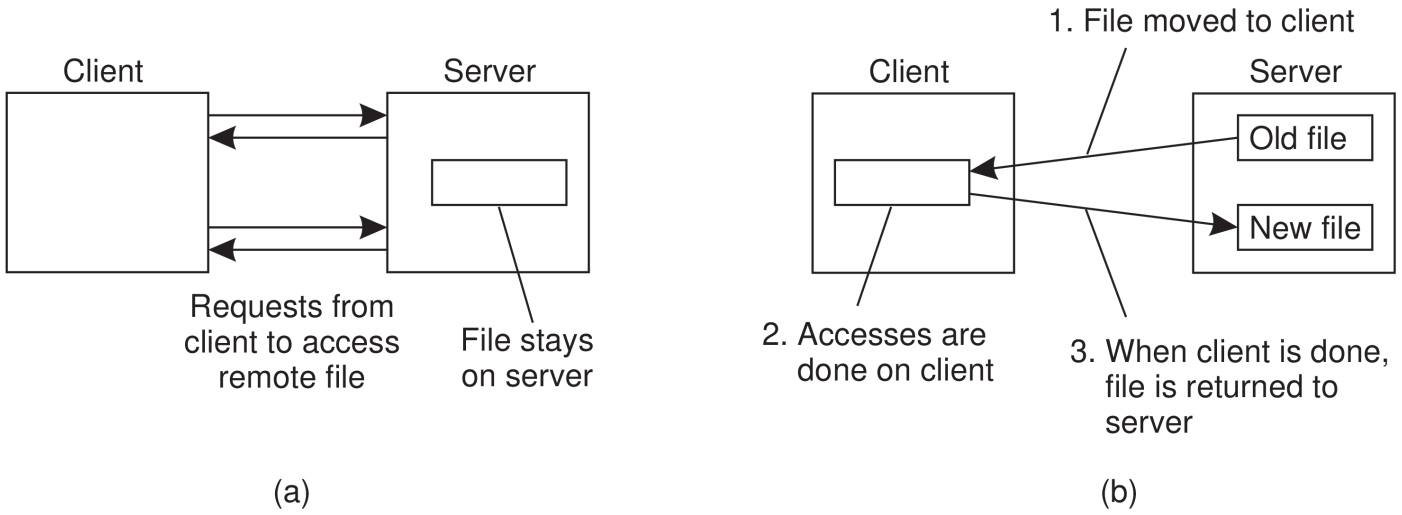
- A server may sometimes need to act as a client.
- In this architecture, traditionally programs that form part of the processing layer are executed by a separate server, but may additionally be partly distributed across the client and server machines.
  - Example: Transaction processing, Websites.



Example: The Network File System

- Sun Microsystem's Network File System (NFS) is widely deployed distributed files system organized like client-server architecture.
- The model underlying NFS and similar systems is that of a **remote file service**.
- Each NFS server provides a standardized view of its local file system.
- Each server supports the same model, regardless the implementation of the file system.

Example: The Network File System

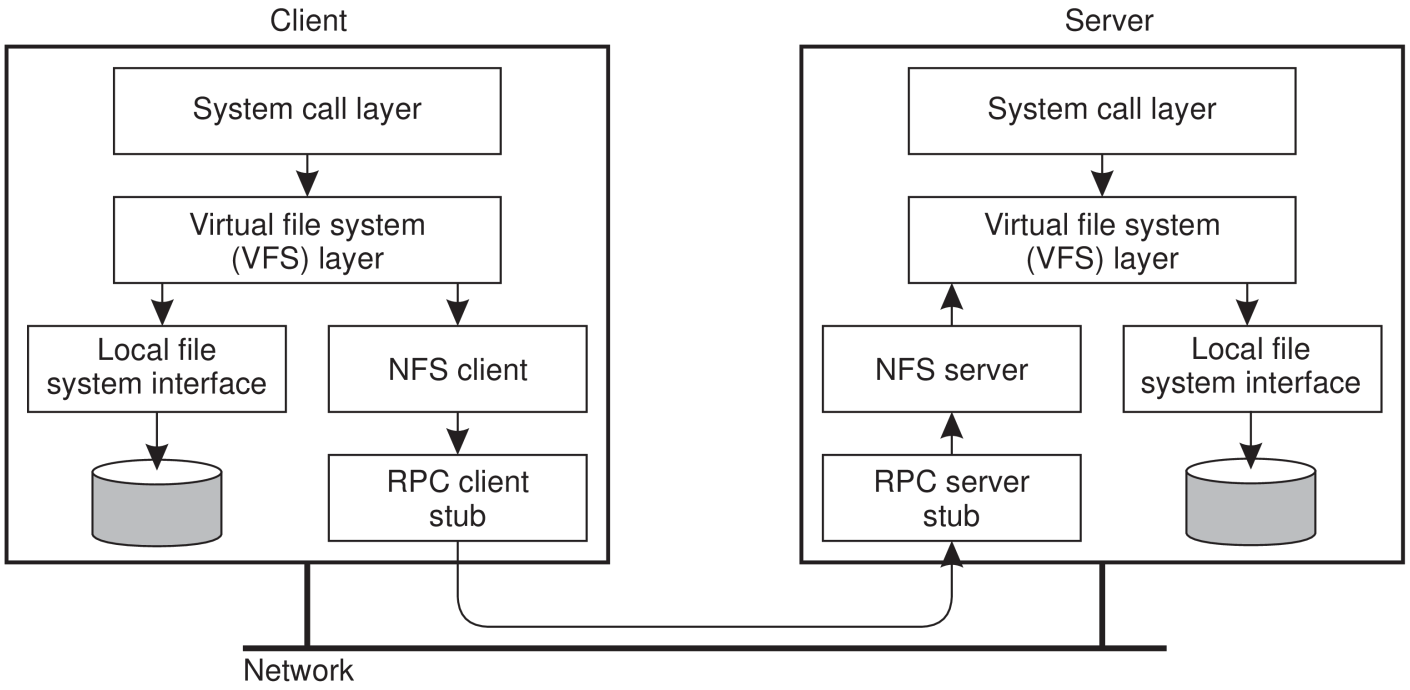


(a) The remote access model.

(b) The upload/download model

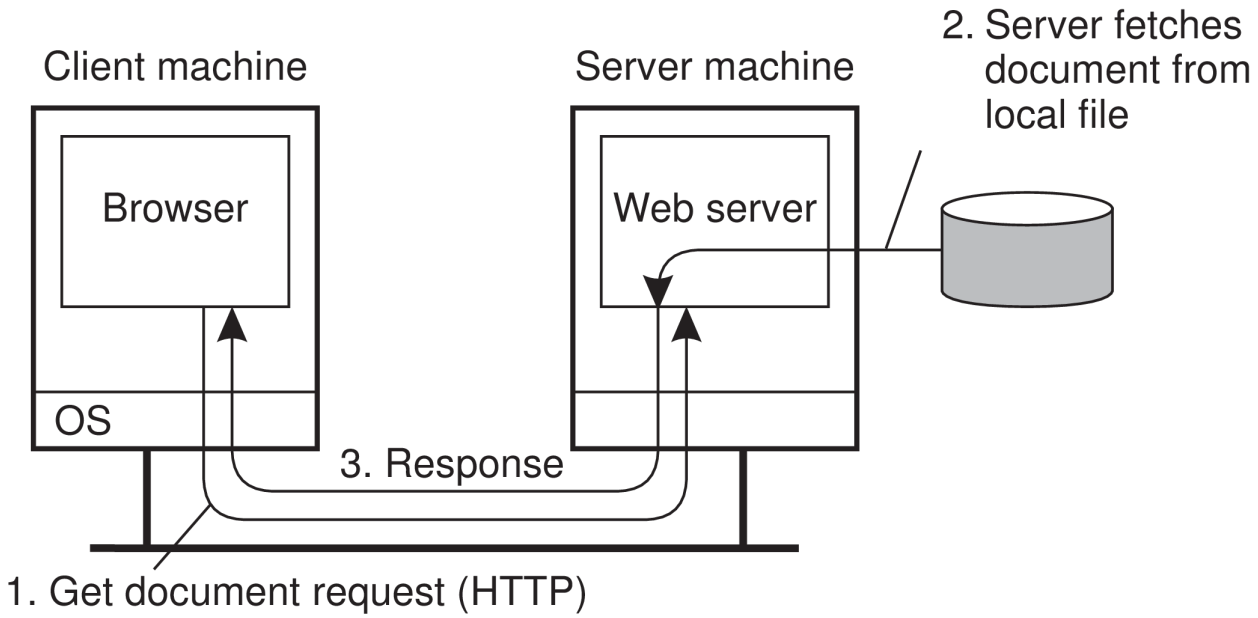
- FTP is a typical upload/download model. The same can be said for systems like Dropbox.

NFS Architecture for Unix Systems



Example: The Web

- Many Web-based systems are still organized as relatively simple client-server architectures.



Wrap-Up + What's Next

In the next lecture, we will discuss symmetrically distributed system architectures including:

- Structured peer-to-peer systems,
- Unstructured peer-to-peer systems,
- Hierarchically organized peer-to-peer networks.